2.1.1. Preguntas

1. ¿Qué cambios realizaría en el diagrama original para representar las sesiones presenciales y virtuales? Justifique las clases y relaciones que incorpore.

Sería modificar la clase `SesionClase`, convirtiéndola en una clase general llamada `Sesion`, donde se mantendrían los atributos y métodos comunes. Despuues , crearía dos subclases, `SesionPresencial` y `SesionVirtual`, que heredarían las características de `Sesion`.

2. ¿Qué atributos y métodos compartirían las sesiones presenciales y virtuales, y cuáles serían propios de cada una? Explique dónde los ubicaría en el modelo y por qué.

Los atributos que compartirían ambas clases serían `fecha`, `duracion` y `realizada`, ya que tanto una sesión presencial como una virtual tienen una fecha, una duración determinada y un registro de si se realizó o no. Estos atributos en una superclase `Sesion`, junto con los métodos `registrarRealizacion` y `obtenerIndicaciones`, para evitar repetirlos en las subclases.

La clase `SesionPresencial` tendría como atributo propio `sala` y como método `obtenerLugar`, porque necesita representar el lugar físico.

Por otro lado, `SesionVirtual` tendría como atributo `enlace` y como método `obtenerAcceso`, porque necesita proporcionar la dirección para ingresar a la clase de manera virtual.

3. ¿Sería adecuado representar las modalidades mediante un atributo de Sesion, sin utilizar herencia? Compare ambas alternativas y justifique cuál elegiría.

No porque tendríamos problemas con la clase donde tendría atributos que no siempre necesitaría . Esto podría generar atributos vacíos y obligarnos a utilizar condiciones para determinar qué comportamiento corresponde a cada modalidad.

4. ¿Es necesario relacionar Asignatura directamente con SesionPresencial y SesionVirtual, o basta con mantener su relación con Sesion? Justifique su respuesta considerando cómo funciona la herencia.

No es necesario, ya que ambas clases heredan de `Sesion`. Por lo tanto, basta con mantener la relación entre `Asignatura` y `Sesion`, porque permite trabajar con ambos tipos de sesiones sin agregar relaciones innecesarias al diagrama.

2.2.1. Preguntas

1. ¿Cómo se utilizaron extends, super(...) y @Override al completar el código? Explique qué función cumple cada uno dentro de la herencia.

El extends establece la relación de la herencia a la clase SessionVirtual llamando los atributos de la clase sesion y el metodo

`super()` se utiliza en la clase hija `SesionVirtual` para llamar al constructor de la clase `Sesion`. En este caso, permite inicializar los atributos `fecha` y `duracion`.

`@Override` se utiliza para indicar al compilador que un método de la subclase está sobrescribiendo un método de su superclase. En este caso, `obtenerIndicaciones()` se redefine para entregar el enlace de acceso a la sesión virtual.

2. ¿Por qué el atributo enlace se incorporó en SesionVirtual y no en Sesion? Justifique su decisión considerando qué características deberían compartir las clases

Porque el enlace es una característica propia de las sesiones virtuales. La clase `Sesion` también tiene una subclase llamada `SesionPresencial`, que no necesita un enlace de acceso.
Por lo tanto, es mejor mantener los atributos comunes en `Sesion` y los atributos específicos en cada subclase.

3. ¿Qué cambios aparecieron en el diagrama después de actualizarlo desde Java? Explique cómo se relacionan con las modificaciones realizadas en el código.

nuevos elementos en la clase `SesionVirtual`: el atributo `enlace`, el constructor `SesionVirtual(fecha, duracion, enlace)` y el método `obtenerIndicaciones()`.

Estos cambios corresponden a las modificaciones realizadas en el código Java, donde se agregó el atributo propio de la clase, utilizamos `super(...)` para llamar al constructor de `Sesion` y `@Override` para sobrescribir el método heredado.